**University of Eloued**

**Series 046**
*Dr. Medini Atmane*
*University of Eloued  Academic Year: 2026/2027*

***Destination: Pro.Atallah amore  and Dr. Ghanaim Zouhira***

---

# *Tree Decision machine*

**The Fundamental Concept of TREE Decision:**
It is an algorithm based on dividing data into branches originating from a main root, where we search at each step for the optimal purity to sort the data. This mixture is measured by a mathematical index called **Gini Impurity**. The algorithm continues to ask sequential logical questions (yes/no) to partition the data and gradually reduce the impurity percentage at each branch, until we ultimately reach stopping points called **Leaves**. At this stage, each leaf is perfectly pure and contains only a single classification category, providing a transparent and automated pathway for making administrative and predictive decisions.

A Decision Tree is a supervised machine learning algorithm that recursively partitions data into mutually exclusive, homogeneous regions. It operates by identifying the optimal feature and threshold to split the dataset, constructing a hierarchical flowchart of decision nodes and terminal leaf nodes that represent the final predicted classes.

**Gini Impurity (Node Evaluation)** To quantify the impurity or mixing of classes within a specific node, the algorithm calculates Gini Impurity. A value of 0 indicates perfect purity (only one class present), while 0.5 indicates maximum mixing for a binary classification problem.

$$Gini = 1 - \sum_{i=1}^{c} (_{P_i})^2$$

Where p_i is the probability of an observation belonging to class i,
and
C is the total number of classes.

Weighted Split Evaluation To evaluate a potential splitting condition, the algorithm calculates the weighted average of the Gini impurities for the resulting left and right child nodes.

$$Gini_{\ Split} = \frac{N_{Left}}{N} Gini_{\ Left} + \frac{N_{right}}{N} Gini_{Right}$$

Where Nis the total number of observations in the parent node, and N_{left} and N_{right} are the count of observations distributed to the respective child nodes.

Gini Gain (Optimization Objective) The fundamental objective of the algorithm is to select the threshold that maximizes the reduction in impurity from the parent node to the child nodes.

$$Gini_{Gain} = Gini_{Parent} - Gini_{Split}$$

**Algorithmic Execution Phases**

1. Compute the baseline Gini impurity of the current parent node.

2. Sort the unique numerical values of each feature and extract the mathematical midpoints between consecutive values to serve as candidate thresholds.
3. Test every candidate threshold across all features by mathematically partitioning the data and computing the resulting Gini split.
4. Isolate the exact feature and threshold combination that yields the lowest possible Gini_{split} (thereby maximizing Gini Gain) and establish it as the decision boundary.
5. Repeat this recursive partitioning process on each new branch until all resulting leaf nodes achieve absolute purity (Gini = 0) or a predefined stopping criterion is triggered.

**Example: (Problem Context: Credit Risk Classification):**
A regional bank is automating its loan approval system using machine learning. The data science team has extracted a simplified historical dataset of six previous loan applicants. The dataset contains two primary numerical features: X_1 (Number of Late Payments) and X_2 (Internal Credit Risk Level). The target variable is the final classification where:

-1 represents a safe, approved client

**and**

+1 represents a high-risk, rejected client.

As a machine learning analyst, your task is to manually construct a binary Decision Tree classifier using the Gini Impurity metric to establish the optimal decision boundary for future loan applicants.

**Exercise Questions:**

**Practical Exercise: Decision Tree Classification Using Python**
**Dataset Reference**
Consider the following dataset representing a simplified credit-risk classification problem. The objective is to develop a Decision Tree classifier that predicts the class of each observation using two input features.

| Observation | Number of Late Payments | Credit Risk Level | Class |
|---|---|---|---|
| A | 1 | 1 | -1 |
| B | 2 | 1 | -1 |
| C | 1 | 2 | -1 |
| D | 4 | 4 | +1 |
| E | 5 | 4 | +1 |
| F | 4 | 5 | +1 |

**Exercise Questions**
**Question 1: Dataset Preparation and Root Node Evaluation**

1. Create a Python DataFrame containing the dataset.
2. Separate the input features (X) from the target variable (y).
3. Write a Python function to calculate the Gini impurity of a given set of class labels.
4. Use the function to calculate the initial Gini impurity of the root node. Interpret the result.

**Question 2: Candidate Threshold Generation**

1. Write Python code to extract the sorted unique values of each feature, (X_1) and (X_2).
2. Implement a function that generates candidate splitting thresholds by calculating the midpoints between consecutive unique values.
3. Display all candidate thresholds generated for each feature.

**Question 3: Evaluating the First Candidate Split**
Consider the splitting condition:

$X_ \leq 1.5$

Write Python code to:

1. Divide the dataset into left and right child nodes according to the splitting condition.
2. Calculate the Gini impurity of each child node.
3. Calculate the weighted Gini impurity of the split.
4. Display the results, including the observations assigned to each child node.

**Question 4: Comparing Candidate Splits and Selecting the Optimal Threshold**

Consider the two candidate splitting conditions:

- ($X_1 \leq 1.5$)
- ($X_1 \leq 3.0$)

1. Write Python code to evaluate the weighted Gini impurity of both splits.
2. Calculate the Gini gain associated with each candidate threshold.
3. Compare the results and identify the optimal root splitting condition.
4. Explain how the algorithm uses Gini impurity to select the best split.

**Question 5: Constructing and Interpreting the Decision Tree**

Based on the optimal splitting condition identified in Question 4:

1. Write Python code to construct a Decision Tree classifier using DecisionTreeClassifier from sklearn.tree.
2. Train the model using the dataset.
3. Display the resulting Decision Tree using plot_tree.
4. Identify the root node, internal nodes, and terminal leaf nodes.
5. Report the predicted class and class purity of each terminal leaf node.
6. Use the trained model to predict the class of a new observation with (X_1=3) and (X_2=3).

**Optional Extension:** Evaluate all candidate thresholds for both features and determine the best split. If multiple thresholds achieve the same minimum weighted Gini impurity, report the tie and explain why it occurs.

## Manually Solution:

The Decision Tree algorithm successfully transforms raw financial data into a direct, automated credit policy by mathematically identifying the optimal threshold for loan approvals. By evaluating all potential splitting conditions using the Gini Impurity metric, the model isolates the exact business rules that perfectly separate safe clients from high-risk applicants.

**Algorithmic Evaluation Results**

| Feature | Candidate Threshold | Weighted Gini Impurity (Gini split) |
|---|---|---|
| X_1 (Late Payments) | ≤1.5 | 0.25 |
| X_1 (Late Payments) | _≤3.0 | **0.00 (Perfect Split)** |
| X_1 (Late Payments) | ≤4.5 | 0.40 |
| X_2 (Risk Level) | ≤1.5 | 0.25 |
| X_2 (Risk Level) | ≤ 3.0 | **0.00 (Perfect Split)** |
| X_2 (Risk Level) | ≤4.5 | 0.40 |

**Extraction of Business Rules** The mathematical optimization reveals that thresholds X_1 ≤3.0 and X_2_≤3.0 both achieve a Gini Impurity of 0.00, meaning they partition the classes with zero errors. Translating the X_1 result into a corporate banking policy means the institution establishes a definitive operational rule: any applicant with 3 or fewer late payments is classified as a safe investment (Class -1), while anyone exceeding 3 late payments is automatically rejected as high-risk (Class +1).

Strategic Value for Management For business administrators and financial managers, this demonstrates how machine learning moves beyond abstract statistics to generate transparent, interpretable, and executable administrative guidelines. The algorithm eliminates human bias and manual guesswork, converting historical data directly into an automated decision-making framework.

**Codding Solution:**

_____Simple Dataset_____

**Full solution of the series:**

https://github.com/atman500/atman500/blob/main/Simple_tree_decission.ipynb

_____Multiple Dataset_____

**Codding Solution of the multiple variabales:**

**Source of data;**

**https://zenodo.org/records/23759666**

**Doi of data:**

**https://doi.org/10.5281/zenodo.23759666**

**Full solution:**

**https://github.com/atman500/atman500/blob/main/Tree_Decision_multiple.ipynb**